

DESAFÍO

Diagnóstico y diseño de una solución inicial con enfoque Big Data

Caso: cadena nacional de supermercados

Propuesta técnica orientada a anticipar peaks de demanda y reducir quiebres de stock

Problema central: durante eventos de alto tráfico (Cyber Monday, ofertas flash y lanzamientos de temporada), la organización no logra anticipar suficientemente la demanda ni reubicar inventario de forma estratégica, pese a disponer de datos de ventas, clima, navegación en app, logística y redes sociales.

Entrega desarrollada para cubrir los criterios de "Completamente logrado" de la escala de apreciación.

1. Caracterización del problema mediante las 5V del Big Data

El caso sí presenta características compatibles con un enfoque Big Data, porque combina múltiples fuentes, distintos ritmos de generación, formatos heterogéneos y una necesidad operacional de reaccionar con rapidez ante cambios de demanda. Las 5V se conectan entre sí: a mayor volumen, velocidad y variedad, aumenta la complejidad de asegurar veracidad; el valor aparece cuando esa complejidad se transforma en decisiones útiles de inventario y demanda.

V	Aplicación al caso	Datos involucrados	Riesgo / necesidad	Decisión habilitada
Volumen	Acumulación masiva de transacciones, navegación, movimientos de inventario, entregas y señales externas.	Tickets POS, eventos de app/web, stock por SKU-tienda, despachos, reclamos, clima.	El enunciado no entrega cantidades. Debe medirse registros/día, GB-TB/día y crecimiento por evento.	Dimensionar almacenamiento, cómputo y retención; priorizar categorías y tiendas de alto impacto.
Velocidad	Parte de los datos se genera continuamente y pierde valor si se procesa demasiado tarde.	Ventas, navegación, stock y logística requieren actualización frecuente; clima y redes sociales pueden actualizarse por ventanas.	Latencia entre ocurrencia y disponibilidad del dato; congestión en eventos de alto tráfico.	Alertas tempranas, reposición, redistribución y ajuste de promociones casi en tiempo real.
Variedad	Conviven datos estructurados, semiestructurados y no estructurados.	Relacionales: ventas e inventario. JSON/logs: app. Texto: reclamos/redes. Externos: clima y calendario de eventos.	Esquemas distintos, claves no homogéneas, granularidades y frecuencias diferentes.	Construir una vista integrada de demanda por tienda, SKU, canal y momento.
Veracidad	La utilidad del sistema depende de la calidad y consistencia de cada fuente.	Stock registrado vs. físico, tickets duplicados, eventos faltantes, geolocalización errónea, texto ambiguo.	Datos incompletos, retrasados, duplicados o inconsistentes pueden sesgar pronósticos.	Reglas de calidad, reconciliación, trazabilidad y scores de confiabilidad antes de usar datos.
Valor	El objetivo no es almacenar más datos, sino mejorar decisiones comerciales y operativas.	Pronósticos, inventario, navegación, sentimiento, clima y logística combinados.	Sin integración y analítica, las fuentes permanecen aisladas y no generan acción coordinada.	Anticipar demanda, reasignar inventario, priorizar reposición y personalizar promociones.

Supuestos de dimensionamiento inicial (propuestos, no entregados por el enunciado): para una cadena nacional conviene iniciar el diseño midiendo, al menos, transacciones y líneas de ticket por día, eventos de navegación por segundo en horas peak, actualizaciones de stock por SKU-tienda, volumen de texto social y GB/TB diarios. Para un piloto se pueden usar rangos de planificación y luego reemplazarlos por métricas reales del negocio antes de dimensionar infraestructura productiva.

2. Propuesta inicial de solución Big Data

2.1 Fuentes de datos y mecanismo de recolección

Fuente	Contenido	Ingesta propuesta	Uso analítico
POS / e-commerce	Tickets, productos, cantidades, precios, descuentos, tienda/canal, timestamp.	CDC o eventos hacia Kafka; carga histórica batch.	Demanda observada, patrones por SKU/tienda, elasticidad y promociones.
App / web	Búsquedas, vistas, carrito, abandono, clics, navegación.	Streaming de eventos (Kafka) y logs batch.	Señales tempranas de intención de compra antes de concretarse la venta.
Inventario	Stock disponible, reservado, en tránsito, roturas, reposición.	CDC desde sistemas operacionales; snapshots periódicos.	Disponibilidad real y capacidad de respuesta ante peaks.
Centros de distribución / transporte	Despachos, tiempos, ETA, capacidad y rutas.	APIs, CDC o archivos batch.	Factibilidad y tiempo de redistribución.
Clima / eventos	Pronóstico, temperatura, lluvia, feriados, eventos comerciales.	APIs externas por intervalos.	Variables exógenas para anticipar cambios de demanda.
Redes sociales / reclamos	Texto, temas, sentimiento, volumen de menciones.	APIs/conectores; micro-batch o streaming.	Detectar señales cualitativas de demanda, desabastecimiento o reputación.

2.2 Arquitectura técnica propuesta

Se propone una arquitectura de procesamiento mixto (streaming + batch) con Apache Spark como componente central. Spark permite procesar grandes volúmenes de manera distribuida, ejecutar transformaciones en memoria y trabajar tanto con lotes históricos como con flujos de datos.

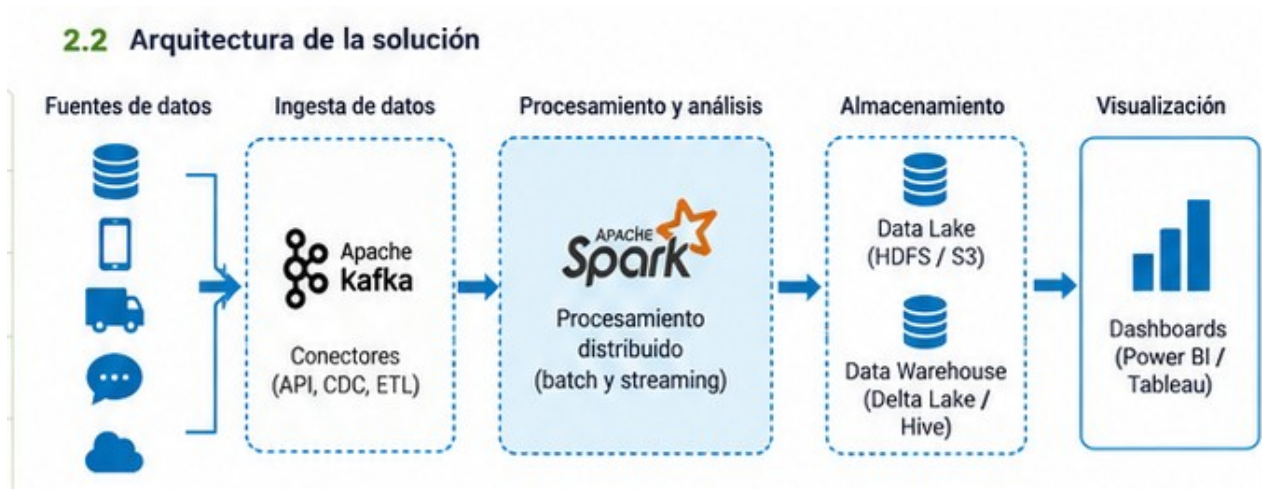


Figura 1. Esquema conceptual de la arquitectura propuesta (ilustrativo).

1. Fuentes	2. Ingesta	3. Procesamiento	4. Almacenamiento	5. Analítica	6. Acción
POS, app, inventario, logística, clima, redes	Kafka + APIs + CDC + batch	Apache Spark / Structured Streaming	Data Lake + capa curada / warehouse	Modelos de demanda + reglas + dashboards	Alertas, reposición, redistribución, promociones

2.3 Tecnologías y justificación

Capa	Tecnología propuesta	Función	Justificación
Ingesta	Apache Kafka + APIs + CDC	Capturar eventos en streaming y cambios de sistemas fuente.	Desacopla productores/consumidores y soporta peaks de eventos.
Procesamiento	Apache Spark + Structured Streaming	ETL/ELT, ventanas temporales, agregaciones, feature engineering y scoring.	Es el componente exigido y unifica batch y streaming con cómputo distribuido.
Almacenamiento	Data Lake en almacenamiento de objetos + tablas transaccionales tipo Delta/Iceberg/Hudi	Conservar datos crudos, limpios y curados.	Permite histórico económico, trazabilidad y evolución de esquemas.
Serving / BI	Data warehouse o motor SQL + Power BI/Tableau	Exponer KPI, alertas y análisis para operaciones.	Facilita consumo por usuarios de negocio.
Orquestación	Airflow o servicio equivalente	Programar pipelines batch, controles y dependencias.	Asegura operación repetible y observabilidad.
Calidad / gobierno	Catálogo + reglas de calidad + lineage	Validar completitud, duplicados, frescura y consistencia.	Responde directamente a la V de Veracidad.

2.4 Tipo de procesamiento

- Streaming: ventas, navegación e inventario para detectar aumentos de demanda y riesgo de quiebre durante eventos de alta intensidad.
- Micro-batch o intervalos cortos: clima, redes sociales y algunas fuentes logísticas cuando no exista necesidad de latencia de segundos.
- Batch: históricos, consolidaciones diarias, entrenamiento/reentrenamiento de modelos, análisis de campañas y cierres.
- Justificación: el problema combina decisiones que requieren reacción rápida con análisis históricos de gran volumen; por ello un enfoque mixto es más viable que una solución exclusivamente batch o exclusivamente streaming.

3. Valor estratégico para la organización

3.1 Mejoras medibles y KPI

Objetivo	KPI	Cómo se calcula	Meta inicial sugerida*	Impacto
Reducir quiebres de stock	Tasa de quiebre	SKU-tiempo sin stock / SKU-tiempo total	-20% a -30% en piloto	Mayor disponibilidad y menos ventas perdidas.
Mejorar pronóstico	MAPE / WAPE	Error absoluto del pronóstico respecto de ventas reales	Reducir error 15%-25% vs. baseline	Mejor reposición y planificación.
Acelerar reacción	Latencia dato → alerta	Tiempo desde evento hasta disponibilidad de señal	<5 min para flujos críticos	Permite intervenir antes de que el quiebre se materialice.
Optimizar inventario	Rotación / días de inventario	Ventas o costo sobre inventario promedio	Mejora progresiva sin elevar quiebres	Menor capital inmovilizado.
Mejorar cliente	Fill rate / reclamos por falta de stock	Pedidos completos / pedidos; reclamos por período	+3 a +5 pp fill rate	Mayor satisfacción y confianza.

* **Metas sugeridas:** son objetivos de piloto propuestos para estructurar la medición; el enunciado no entrega una línea base ni valores reales. Deben recalibrarse con datos históricos de la organización.

3.2 Argumentación estratégica (máx. 250 palabras)

La solución propuesta transforma datos hoy dispersos en una capacidad operacional para anticipar demanda y actuar antes de que se produzca un quiebre de stock. Al integrar ventas, navegación, inventario, logística, clima y señales sociales, Apache Spark puede procesar información histórica y flujos recientes para generar pronósticos y alertas por tienda, categoría y producto. El valor estratégico se materializa en tres frentes. Primero, mejora la disponibilidad de productos y reduce ventas perdidas mediante reposición y redistribución más oportunas. Segundo, disminuye ineficiencias de inventario al equilibrar mejor exceso y faltantes entre tiendas y centros de distribución. Tercero, entrega una base analítica común para coordinar operaciones, comercial y atención al cliente. El éxito debe medirse con KPI como tasa de quiebre de stock, WAPE/MAPE del pronóstico, fill rate, latencia de las alertas, rotación de inventario y reclamos asociados a falta de productos. La arquitectura mixta batch-streaming es coherente con el negocio: el histórico permite aprender patrones y entrenar modelos, mientras que el streaming permite reaccionar durante eventos de alta demanda. Así, el enfoque Big Data no se limita a almacenar información, sino que convierte la velocidad y diversidad de los datos en decisiones medibles y de impacto directo en ventas, eficiencia operativa y experiencia del cliente.

4. Conclusión y recomendación de implementación

El problema es adecuado para una solución Big Data porque presenta simultáneamente las 5V y requiere integrar fuentes de naturalezas distintas. La propuesta recomendada es comenzar con un piloto acotado a algunas tiendas, categorías y un evento de alto tráfico. En ese piloto deben establecerse líneas base de quiebre, error de pronóstico, latencia y rotación; validar la calidad de los datos; desplegar ingesta y procesamiento Spark; y comparar los KPI antes/después. Una vez demostrada la mejora, la arquitectura puede escalarse progresivamente al resto de la cadena.